

Assignment 3 evidence

Yihao Yin (yihaoyin)

generated 2026-09-16 22:29 ????? on Windows-11-10.0.26200-SP0, polars 1.44.2

project HW3

evidence script sha256 1767552b768b078e807883505883ca1389f3909c8043fb3e25b46821bb31ff3f

Score 5.00 / 5 automatic, out of 6 for the assignment.

The last 1 point is REPORT.md, which your TA reads. This script does not score it.

Summary

group	score	checks	decided by
Collection	0.75 / 0.75	4/4 passed	script
Decode and reshape	1.00 / 1.00	5/5 passed	script
Validate	1.00 / 1.00	4/4 passed	script
Dedup and watermark	1.25 / 1.25	4/4 passed	script
Grid and windows	1.00 / 1.00	6/6 passed	script
REPORT.md	? / 1.00	8/8 passed	your TA

automatic total	5.00 / 5.00	of 6 for the assignment	

What this script found

```
raw capture  raw\stream-2026-09-16.ndjson
tidy         processed\tidy.parquet
quarantine   processed\quarantine.parquet
grid         processed\grid.parquet
watermark    results\watermark.csv
windows      results\windows.csv
REPORT       REPORT.md
              574 telemetry messages, 30,422 readings, 28.9 plant hours
```

Collection 0.75 / 0.75

```
[PASS] the raw NDJSON holds the birth message
[PASS] at least 20 plant hours of event time
       28.9 plant hours
[PASS] receivedAt never goes backwards
[PASS] the raw file is as the collector wrote it
```

Decode and reshape 1.00 / 1.00

```
[PASS] tidy.parquet has exactly the eight columns, with their types
[PASS] the decode is a Polars LazyFrame collected once
[PASS] the three clocks are distinct and populated
[PASS] more than one event_time per server_time
[PASS] the record count reconciles with the raw file
       24,001 rows against 24,004 distinct Good readings in the raw file
```

Validate 1.00 / 1.00

```
[PASS] a pandera schema generated from the birth tags
[PASS] validation runs with lazy=True
[PASS] Bad and Uncertain are in quarantine.parquet and not in tidy.parquet
[PASS] the report's status and quarantine counts reconcile
```

Dedup and watermark 1.25 / 1.25

```
[PASS] (tag, event_time) is unique in tidy.parquet
[PASS] the duplicate count matches the script's
       the script counts 6,362 (dedup first) or 6,333 (quarantine first); your tables imply 6,333
[PASS] watermark.csv sweeps several allowances, with completeness and still waiting
[PASS] the report compares event time with processing time
```

Grid and windows 1.00 / 1.00

```

[PASS] a regular 180 s grid, one column per tag, no empty cells
[PASS] a record of which cells were filled
[PASS] the empty-cell counts before filling reconcile
       rebuilt: continuous 192 (0.98%), analyser 6,340 (57.93%)
[PASS] wide 30-minute windows.csv that the script recomputes from the grid
       59 of 59 windows matched, 100.0% of cells agree
[PASS] the incomplete last window is handled
       a count or completeness column
[PASS] the disturbance is given an interval and named tags

```

REPORT.md 1.00 point, read by your TA

```

[PASS] section: the three clocks
[PASS] section: what you collected
[PASS] section: what you filled
[PASS] section: the watermark
[PASS] section: event time against processing time
[PASS] section: the disturbance
[PASS] section: what .explain() showed
[PASS] the generative-AI use statement

```

Each line is decided from the output below, not asserted.

Worth a look

Not scored here. These are for whoever reads the report.

- the collector received nothing for 46 s at one point; anything published then is simply not in the file

Transcript

parse the raw capture

```
$ read 1 file(s) [ok, 0.0s]
```

```
parsed D:\Lecture\System & Tool Chains\Week3\HW3\raw\stream-2026-09-16.ndjson with json.loads, one line
at a time
```

```

575 messages (574 telemetry, 1 birth, 0 event)
30,422 readings
129 messages marked isHistorical

30,337 Good
70 Uncertain_SensorNotAccurate
15 Bad_DeviceFailure

24,060 distinct (tag, event_time) over all readings
24,004 distinct (tag, event_time) over Good readings
6,362 duplicates if you deduplicate first
6,333 duplicates if you quarantine first

```

```

event time 2029-07-20T02:37:54.016Z to 2029-07-21T07:34:54.016Z
28.95 plant hours
received 2026-09-17T01:03:51.378Z to 2026-09-17T01:15:51.079Z
largest gap between two arrivals 46.3 s

```

```

lines whose receivedAt is earlier than the line before: 0
lines not in the collector's exact format: 0
lines that are not JSON: 0; blank lines: 0

```

read the tidy table

```
$ polars.read_parquet(D:\Lecture\System & Tool Chains\Week3\HW3\processed\tidy.parquet) [ok, 0.0s]
```

```
D:\Lecture\System & Tool Chains\Week3\HW3\processed\tidy.parquet: 24,001 rows, 8 columns
```

```

tag          String          ok
value        Float64         ok
status       String         ok
event_time   Datetime(time_unit='ms', time_zone='UTC') ok

```

```

server_time      Datetime(time_unit='ms', time_zone='UTC') ok
received_at      Datetime(time_unit='ms', time_zone='UTC') ok
seq              Int64                                     ok
is_historical     Boolean                                  ok

rows sharing a (tag, event_time) with another row: 0
(tag, event_time) pairs that are not in the raw capture: 0
rows whose status is not Good: 0

null clock values: 0
rows where event_time differs from server_time: 22.7%
rows where received_at equals server_time: 0.0%
server_time values carrying more than one event_time: 53 of 443

```

read the quarantine table

```
$ polars.read_parquet(D:\Lecture\System & Tool Chains\Week3\HW3\processed\quarantine.parquet) [ok, 0.0s]
```

```
D:\Lecture\System & Tool Chains\Week3\HW3\processed\quarantine.parquet: 88 rows; columns tag, value,
status, event_time, server_time, received_at, seq, is_historical, reason
```

```

Uncertain_* rows 70; Bad_* rows 15
reasons:
    70 status: Uncertain_SensorNotAccurate
    15 status: Bad_DeviceFailure
    3  schema: value_in_unit_range

```

read the grid

```
$ polars.read_parquet(D:\Lecture\System & Tool Chains\Week3\HW3\processed\grid.parquet) [ok, 0.0s]
```

```

D:\Lecture\System & Tool Chains\Week3\HW3\processed\grid.parquet: 576 rows, 107 columns; time column
event_time
spacing between rows: 180 s
birth tags missing as columns: 0
empty cells after filling: 0
per-tag fill-mask columns: 53

```

```

empty cells before filling, rebuilt from the tidy table over this grid:
continuous 34 tags      192 of 19,584 (0.98%) your mask marks 192
analyser    19 tags      6,340 of 10,944 (57.93%) your mask marks 6,340

```

recompute the windows

```
$ group grid.parquet into 30-minute buckets [ok, 0.0s]
```

```

D:\Lecture\System & Tool Chains\Week3\HW3\results\windows.csv: 59 rows, 53 tag columns, window column
window_start
most common spacing between windows: 30 min
windows recomputed from the grid: 59 (the last holds 2 of 10 samples)
your windows matched to them: 59
cells agreeing to 1e-5: 100.0%
a column recording the count or completeness of each window: yes

```

read the watermark sweep

```
$ polars.read_csv(D:\Lecture\System & Tool Chains\Week3\HW3\results\watermark.csv) [ok, 0.0s]
```

```

D:\Lecture\System & Tool Chains\Week3\HW3\results\watermark.csv: 9 rows
allowance column allowance_plant_min; completeness column completeness_pct; still-waiting column
still_waiting

```

```

allowance_plant_min,wall_clock_s,completeness_pct,dropped_as_late,still_waiting
0,0.0,77.684,5356,0
15,6.25,77.684,5356,202
30,12.5,78.163,5241,419
45,18.75,82.538,4191,622
60,25.0,87.609,2974,837
90,37.5,96.646,805,1256
120,50.0,100.0,0,1676
180,75.0,100.0,0,2514

```

240,100.0,100.0,0,3353

read the code

11 code files: src\clean.py, src\contract.py, src\decode.py, src\disturbance.py, src\gate_test.py, src\grid.py, src\idempotence.py, src\pipeline.py, src\schema.py, src\stream.py, src\windows.py

lazy decode

src\decode.py: scan or .lazy(), 1 .collect() call(s)
.explain() called: yes; infer_schema_length=None or an explicit schema: yes

pandera

src\schema.py
schema reads the birth tags: yes; lazy=True: yes
distinct tag names typed as string literals: 1

dedup keyed on seq: no

Code: src\decode.py

```
"""Stage 1: decode the raw NDJSON into one row per reading, lazily.
```

```
The whole stage is one Polars query plan: scan, filter to telemetry, pull the  
message fields out of the payload struct, explode the metrics list, unnest the  
metric struct, parse the three clocks, cast. It is collected exactly once.  
"""
```

```
from __future__ import annotations
```

```
import polars as pl
```

```
from contract import RAW_GLOB, RESULTS, ROOT, TELEMETRY
```

```
STAMP = "%Y-%m-%dT%H:%M:%S%.3fZ"
```

```
TIDY_COLUMNS = ["tag", "value", "status", "event_time", "server_time", "received_at",  
                "seq", "is_historical"]
```

```
def utc(expr: pl.Expr) -> pl.Expr:  
    return expr.str.to_datetime(STAMP, time_unit="ms", time_zone="UTC", strict=True)
```

```
def plan() -> pl.LazyFrame:  
    payload = pl.col("payload")  
    metric = pl.col("metric")  
    return (  
        # Birth and telemetry payloads have different fields; reading every line  
        # to infer the schema keeps the fields that first appear late.  
        pl.scan_ndjson(str(ROOT / RAW_GLOB), infer_schema_length=None)  
        .with_row_index("line")  
        .filter(pl.col("topic") == TELEMETRY)  
        .select(  
            "line",  
            pl.col("receivedAt").alias("received_raw"),  
            payload.struct.field("seq").alias("seq"),  
            payload.struct.field("timestamp").alias("server_raw"),  
            payload.struct.field("isHistorical").alias("is_historical"),  
            payload.struct.field("metrics").alias("metric"),  
        )  
        .explode("metric", empty_as_null=True)  
        .select(  
            "line",  
            metric.struct.field("name").alias("tag"),  
            metric.struct.field("value").cast(pl.Float64).alias("value"),  
            metric.struct.field("statusCode").alias("status"),  
            utc(metric.struct.field("sourceTimestamp")).alias("event_time"),  
            utc(pl.col("server_raw")).alias("server_time"),
```

```

        utc(pl.col("received_raw")).alias("received_at"),
        pl.col("seq").cast(pl.Int64),
        pl.col("is_historical").cast(pl.Boolean),
    )
)

```

```

def decode(show_plan: bool = True) -> pl.DataFrame:
    """Every reading in the capture, in arrival order, with its line number."""
    query = plan()
    explained = query.explain()
    RESULTS.mkdir(exist_ok=True)
    (RESULTS / "explain.txt").write_text(explained + "\n", encoding="utf-8")
    if show_plan:
        print("optimised plan:\n" + explained + "\n")
    return query.collect()

if __name__ == "__main__":
    frame = decode()
    print(frame.head())
    print(frame.schema)

```

Code: src\schema.py

```

from __future__ import annotations

from datetime import datetime, timedelta

import pandas as pd
import pandera.pandas as pa

from contract import Contract

# Wide enough that a plausible reading passes and an impossible one does not.
UNIT_RANGE = {
    "Mole %": (0.0, 100.0),
    "%": (0.0, 100.0),
    "Deg C": (0.0, 200.0),
    "kPa gauge": (0.0, 4000.0),
    "kscmh": (0.0, 200.0),
    "kg/hr": (0.0, 20000.0),
    "m3/hr": (0.0, 200.0),
    "kW": (0.0, 2000.0),
}

# OPC UA status codes the publisher is documented to send.
STATUSES = {"Good", "Uncertain_SensorNotAccurate", "Bad_DeviceFailure"}

# How far behind the first arrival an event may legitimately be: a replayed
# (historical) message can carry readings a few plant hours old.
LOOKBACK = timedelta(hours=6)
LOOKAHEAD = timedelta(hours=1)

STAMP = pd.DatetimeTZDtype("ms", "UTC")

def build_schema(birth: dict, window: tuple[datetime, datetime]) -> pa.DataFrameSchema:
    tags = birth["tags"]
    names = [t["name"] for t in tags]
    unit = {t["name"]: t["unit"] for t in tags}
    unknown = sorted(set(unit.values()) - set(UNIT_RANGE))
    assert not unknown, f"the birth message declares a unit with no range here: {unknown}"
    low = {name: UNIT_RANGE[u][0] for name, u in unit.items()}
    high = {name: UNIT_RANGE[u][1] for name, u in unit.items()}
    start, end = window

```

```

def value_in_unit_range(frame: pd.DataFrame) -> pd.Series:
    # One range per unit, looked up per row: a whole-frame check.
    # A missing value is not_nullable's failure, not this check's.
    lo = frame["tag"].map(low)
    hi = frame["tag"].map(high)
    return frame["value"].isna() | frame["value"].between(lo, hi)

def measured_before_published(frame: pd.DataFrame) -> pd.Series:
    return frame["event_time"] <= frame["server_time"]

# A built-in check reports its 'error' text in failure_cases, not its name.
columns = {
    "tag": pa.Column(str, pa.Check.isin(names, error="tag_declared_in_birth")),
    "value": pa.Column(float, nullable=False),
    "status": pa.Column(str, pa.Check.isin(STATUSES, error="status_known")),
    "event_time": pa.Column(STAMP, pa.Check.in_range(start, end, error="event_time_in_collection_window"
)),
    "server_time": pa.Column(STAMP),
    "received_at": pa.Column(STAMP),
    "seq": pa.Column(int, pa.Check.in_range(0, 255, error="seq_is_one_byte")),
    "is_historical": pa.Column(bool),
}
checks = [
    pa.Check(value_in_unit_range, name="value_in_unit_range"),
    pa.Check(measured_before_published, name="event_time_at_or_before_server_time"),
]
return pa.DataFrameSchema(columns, checks=checks, strict=True, coerce=False)

def collection_window(c: Contract, received: pd.Series) -> tuple[datetime, datetime]:
    first = c.plant_time(received.min().to_pydatetime())
    last = c.plant_time(received.max().to_pydatetime())
    return first - LOOKBACK, last + LOOKAHEAD

def failures(schema: pa.DataFrameSchema, frame: pd.DataFrame) -> tuple[pd.Series, pd.DataFrame]:
    """Validate everything at once (lazy=True) and return, per row index, the
    names of the checks that row failed, plus pandera's failure-case table."""
    try:
        schema.validate(frame, lazy=True)
    except pa.errors.SchemaErrors as err:
        cases = err.failure_cases
        rows = cases.dropna(subset=["index"]).copy()
        rows["index"] = rows["index"].astype(int)
        reasons = (rows.groupby("index")["check"]
                    .agg(lambda s: ";".join(sorted(set(map(str, s)))))
                    .reset_index())
        return reasons, cases
    return pd.Series(dtype=str), pd.DataFrame()

```

Code: src\clean.py

```

from __future__ import annotations

import json

import pandas as pd

from contract import PROCESSED, RESULTS, Contract, contract
from decode import TIDY_COLUMNS, decode
from schema import build_schema, collection_window, failures

KEY = ["tag", "event_time"]

def clean(readings: pd.DataFrame, birth: dict, window) -> tuple[pd.DataFrame, pd.DataFrame, dict]:
    """Quarantine, deduplicate, validate. Returns (tidy, quarantine, counts);
    tidy still carries 'line' (arrival order) for the lateness stage."""

```

```

# 1. The device's verdict, one delivery at a time. A republished analyser
#     value can arrive Good in one message and Uncertain or Bad in another.
flagged = readings["status"] != "Good"
by_status = readings[flagged].assign(reason="status: " + readings.loc[flagged, "status"])
good = readings[~flagged]

# 2. One row per measurement: what was measured, and when. seq is one byte
#     and wraps every 256 messages, so it is not part of the key. Good
#     deliveries of one key carry the same value; the first arrival is kept.
measurements = good.drop_duplicates(KEY, keep="first")

# 3. The schema, every check at once.
schema = build_schema(birth, window)
reasons, cases = failures(schema, measurements[TIDY_COLUMNS])
if len(cases) and cases["index"].isna().any():
    raise SystemExit("schema failures not tied to a row (a column is missing or mistyped):\n"
                    + cases[cases["index"].isna()].to_string())
by_schema = measurements.loc[reasons.index].assign(reason="schema: " + reasons)
tidy = measurements.drop(index=reasons.index)

quarantine = pd.concat([by_status, by_schema])
counts = {
    "messages": int(readings["line"].nunique()),
    "readings": len(readings),
    "status": readings["status"].value_counts().to_dict(),
    "good_deliveries": len(good),
    "duplicates_removed": len(good) - len(measurements),
    "good_measurements": len(measurements),
    "schema_failure_cases": cases["check"].value_counts().to_dict() if len(cases) else {},
    "schema_rejected": len(by_schema),
    "tidy_rows": len(tidy),
    "quarantine_rows": len(quarantine),
    "quarantine_reasons": quarantine["reason"].value_counts().to_dict(),
    "window_plant": [str(window[0]), str(window[1])],
}

return tidy, quarantine, counts

def write(tidy: pd.DataFrame, quarantine: pd.DataFrame, counts: dict) -> None:
    PROCESSED.mkdir(exist_ok=True)
    RESULTS.mkdir(exist_ok=True)
    (tidy[TIDY_COLUMNS].sort_values(["event_time", "tag"], kind="stable")
     .to_parquet(PROCESSED / "tidy.parquet", index=False))
    (quarantine[TIDY_COLUMNS + ["reason"]].sort_values(["event_time", "tag", "received_at"], kind="stable")
     .to_parquet(PROCESSED / "quarantine.parquet", index=False))
    (RESULTS / "clean_counts.json").write_text(json.dumps(counts, indent=2) + "\n", encoding="utf-8")

def run(c: Contract, readings: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame, dict]:
    tidy, quarantine, counts = clean(readings, c.birth, collection_window(c, readings["received_at"]))
    write(tidy, quarantine, counts)
    return tidy, quarantine, counts

def main() -> tuple[pd.DataFrame, pd.DataFrame, dict]:
    return run(contract(), decode(show_plan=False).to_pandas())

if __name__ == "__main__":
    _, _, counts = main()
    print(json.dumps(counts, indent=2))

```

Code: src\contract.py

```

from __future__ import annotations

import json

```

```

from datetime import datetime, timedelta
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
RAW_GLOB = "raw/stream-*.ndjson"
PROCESSED = ROOT / "processed"
RESULTS = ROOT / "results"

TELEMETRY = "plant/tep/telemetry"
BIRTH = "plant/tep/birth"

SAMPLE = timedelta(seconds=180)
WINDOW = "30m"

def raw_files() -> list[Path]:
    files = sorted(ROOT.glob(RAW_GLOB))
    if not files:
        raise SystemExit(f"no {RAW_GLOB} under {ROOT}; run the collector first")
    return files

def load_birth() -> dict:
    """The first birth message in the capture. The broker retains it, so the
    collector always writes it as the first line of a run."""
    for path in raw_files():
        with path.open(encoding="utf-8") as handle:
            for line in handle:
                if f'"topic": "{BIRTH}"' in line[:80]:
                    return json.loads(line)["payload"]
    raise SystemExit("no birth message in the capture")

class Contract:
    def __init__(self, birth: dict):
        self.birth = birth
        self.tags: list[dict] = birth["tags"]
        self.names: list[str] = [t["name"] for t in self.tags]
        self.unit: dict[str, str] = {t["name"]: t["unit"] for t in self.tags}
        self.analysers: list[str] = [t["name"] for t in self.tags
                                     if t.get("analyserIntervalHours") is not None]
        self.continuous: list[str] = [n for n in self.names if n not in self.analysers]
        self.acceleration: float = float(birth["accelerationFactor"])
        self.plant_epoch: datetime = datetime.fromisoformat(birth["plantEpoch"].replace("Z", "+00:00"))
        self.sample_seconds: int = int(birth["sampleIntervalSeconds"])

    def plant_time(self, wall):
        """receivedAt_plant = plantEpoch + (receivedAt - plantEpoch) * accelerationFactor

        Works on one datetime or on a pandas Series of them."""
        return self.plant_epoch + (wall - self.plant_epoch) * self.acceleration

def contract() -> Contract:
    return Contract(load_birth())

```

Code: src\disturbance.py

```

from __future__ import annotations

import json

import numpy as np
import pandas as pd

from contract import RESULTS, Contract, contract

```

```

WINDOW = pd.Timedelta(minutes=30)
QUIET_Z = 3.0          # a window is quiet when no tag is beyond this many sigmas
CUT_SD = 3.0           # a window stands out when its score exceeds quiet mean + CUT_SD * quiet sd
SINGLE_TAG_Z = 4.0      # outside an episode, one tag this far out is reported as ambiguous

def score_windows(windows: pd.DataFrame, c: Contract) -> tuple[pd.DataFrame, pd.DataFrame, dict]:
    """Score each complete window against a baseline built from quiet windows only.
    Returns (per-window table, z-scores, method facts)."""
    full = windows[windows["complete"]].set_index("window_start")[c.continuous]
    flat = [t for t in full.columns if full[t].std() == 0]
    full = full.drop(columns=flat)

    # Pass 1: median and MAD over every complete window. Both ignore a few
    # extreme windows, so the disturbance cannot hide itself in this yardstick.
    med = full.median()
    mad = ((full - med).abs().median() * 1.4826).replace(0, np.nan)
    robust = (full - med) / mad
    quiet = ~(robust.abs() > QUIET_Z).any(axis=1)

    # Pass 2: mean and standard deviation from the quiet windows only.
    sd = full[quiet].std().replace(0, np.nan)
    z = ((full - full[quiet].mean()) / sd).dropna(axis=1, how="all")
    score = z.abs().mean(axis=1)
    cut = float(score[quiet].mean() + CUT_SD * score[quiet].std())

    table = pd.DataFrame({
        "score": score.round(4),
        "quiet": quiet,
        "standout": score > cut,
        "max_abs_z": z.abs().max(axis=1).round(3),
        "top_tags": z.abs().apply(lambda r: "; ".join(f"{t} {z.loc[r.name, t]:+.1f}"
                                                    for t in r.nlargest(3).index), axis=1),
    })
    facts = {
        "complete_windows": len(full),
        "incomplete_windows_excluded": int((~windows["complete"]).sum()),
        "tags_scored": z.shape[1],
        "tags_excluded_no_spread": flat + sorted(set(full.columns) - set(z.columns)),
        "quiet_windows": int(quiet.sum()),
        "quiet_score_mean": round(float(score[quiet].mean()), 4),
        "quiet_score_sd": round(float(score[quiet].std()), 4),
        "cut": round(cut, 4),
    }
    return table, z, facts

def runs(flags: pd.Series) -> list[pd.Index]:
    """Consecutive True stretches, as lists of window starts."""
    ids = (flags != flags.shift()).cumsum()
    return [group.index for _, group in flags[flags].groupby(ids[flags])]

def episodes(table: pd.DataFrame, z: pd.DataFrame, c: Contract) -> tuple[list[dict], list[dict]]:
    desc = {t["name"]: t["description"] for t in c.tags}
    moving = table["max_abs_z"] > QUIET_Z
    starts = list(table.index)
    found, covered = [], set()
    for core in runs(table["standout"]):
        # Widen the core over neighbouring windows where some tag is still
        # beyond QUIET_Z: that is the onset before and the recovery after.
        lo, hi = starts.index(core[0]), starts.index(core[-1])
        while lo > 0 and moving.iloc[lo - 1]:
            lo -= 1
        while hi < len(starts) - 1 and moving.iloc[hi + 1]:
            hi += 1
        span = starts[lo:hi + 1]

```

```

covered.update(span)
peak = table.loc[core, "score"].idxmax()
biggest = z.loc[span].abs().max().nlargest(5)
found.append({
    "disturbed_from": str(span[0]),
    "recovered_by": str(span[-1] + WINDOW),
    "standout_windows": [str(t) for t in core],
    "onset_or_recovery_windows": [str(t) for t in span if t not in core],
    "starts_at_first_complete_window": lo == 0,
    "peak_window": str(peak),
    "peak_score": float(table.loc[peak, "score"]),
    "tags": [{"tag": t, "description": desc[t], "max_abs_z": round(float(v), 1),
               "z_at_peak": round(float(z.loc[peak, t]), 1)} for t, v in biggest.items()],
})

ambiguous = []
outside = (table["max_abs_z"] > SINGLE_TAG_Z) & ~table.index.isin(list(covered))
for span in runs(outside):
    tags = z.loc[span].abs().max()
    tags = tags[tags > SINGLE_TAG_Z].sort_values(ascending=False)
    ambiguous.append({
        "from": str(span[0]),
        "to": str(span[-1] + WINDOW),
        "scores": [float(table.loc[t, "score"]) for t in span],
        "tags": [{"tag": t, "description": desc[t],
                   "z_by_window": [round(float(z.loc[w, t]), 1) for w in span]} for t in tags.index],
    })
return found, ambiguous

```

```

def main() -> dict:
    c = contract()
    windows = pd.read_csv(RESULTS / "windows.csv", parse_dates=["window_start"])
    table, z, facts = score_windows(windows, c)
    found, ambiguous = episodes(table, z, c)
    summary = {"method": {"quiet_z": QUIET_Z, "cut_sd": CUT_SD, "single_tag_z": SINGLE_TAG_Z, **facts},
               "episodes": found, "ambiguous": ambiguous}

    RESULTS.mkdir(exist_ok=True)
    table.join(z.round(3).add_prefix("z_")).reset_index().to_csv(RESULTS / "disturbance.csv", index=False)
    (RESULTS / "disturbance.json").write_text(json.dumps(summary, indent=2) + "\n", encoding="utf-8")
    return {"table": table, "z": z, "summary": summary}

if __name__ == "__main__":
    out = main()
    print(json.dumps(out["summary"], indent=2))

```

Code: src\gate_test.py

```

from __future__ import annotations

import numpy as np
import pandas as pd

from contract import PROCESSED, RESULTS, contract
from decode import TIDY_COLUMNS
from schema import UNIT_RANGE, build_schema, collection_window, failures

COPIED = 300

def break_rows(frame: pd.DataFrame, unit: dict[str, str]) -> dict[int, str]:
    """Break one row per failure the assignment names. Returns {row: what was done}."""
    broken = {}

    row = 10

```

```

frame.loc[row, "tag"] = "XMEAS_99_not_in_birth"
broken[row] = "undeclared tag"

row = 60
high = UNIT_RANGE[unit[frame.loc[row, "tag"]]][1]
frame.loc[row, "value"] = high * 10
broken[row] = f"value 10x the top of its unit range ({high * 10:g})"

row = 110
frame.loc[row, "value"] = np.nan
broken[row] = "missing value on a Good reading"

row = 160
frame.loc[row, "status"] = "Questionable"
broken[row] = "unknown status code"

row = 210
frame.loc[row, "event_time"] = frame.loc[row, "event_time"] - pd.DateOffset(years=1)
broken[row] = "event_time one year early"

return broken

```

```

def main() -> str:
    c = contract()
    tidy = pd.read_parquet(PROCESSED / "tidy.parquet")
    schema = build_schema(c.birth, collection_window(c, tidy["received_at"]))

    copy = tidy.sample(n=COPIED, random_state=6763).reset_index(drop=True)[TIDY_COLUMNS]
    out = [f"gate test: {COPIED} clean rows copied from processed/tidy.parquet"]

    reasons, _ = failures(schema, copy)
    out.append(f" before breaking anything: {len(reasons)} rows fail the schema")

    before = copy.copy()
    broken = break_rows(copy, c.unit)
    out.append(f" broke {len(broken)} rows on purpose, then validated with lazy=True\n")

    reasons, cases = failures(schema, copy)
    unrowed = cases[cases["index"].isna()] if len(cases) else cases
    if len(unrowed):
        out.append("failures not tied to a row:\n" + unrowed.to_string() + "\n")

    out.append("which check caught which rows (from SchemaErrors.failure_cases):")
    by_check = cases.dropna(subset=["index"]).groupby("check")["index"].agg(
        lambda s: sorted(set(s.astype(int))))
    for check, rows in by_check.items():
        out.append(f" {check:<34} rows {rows}")

    out.append("\nrow by row:")
    for row, what in broken.items():
        was, now = before.loc[row], copy.loc[row]
        changed = [col for col in TIDY_COLUMNS if not (pd.isna(was[col]) and pd.isna(now[col])) and was[col]
            != now[col]]
        detail = ", ".join(f"{col}: {was[col]} -> {now[col]}" for col in changed)
        out.append(f" row {row:>3} {what}\n          {detail}\n"
            f"          caught by: {reasons.get(row, 'NOTHING')}")

    missed = [row for row in broken if row not in reasons.index]
    false_alarms = [row for row in reasons.index if row not in broken]
    out.append(f"\nbroken rows caught: {len(broken) - len(missed)} of {len(broken)}"
        + (f" (missed {missed})" if missed else ""))
    out.append(f"clean rows flagged: {len(false_alarms)}" + (f" {false_alarms}" if false_alarms else ""))
    report = "\n".join(out)

    RESULTS.mkdir(exist_ok=True)
    (RESULTS / "gate_test.txt").write_text(report + "\n", encoding="utf-8")

```

```

if missed or false_alarms:
    raise SystemExit(report + "\n\nGATE TEST FAILED")
return report

```

```

if __name__ == "__main__":
    print(main())

```

Code: src\grid.py

```

from __future__ import annotations

```

```

import json

```

```

import pandas as pd

```

```

from clean import clean
from contract import PROCESSED, RESULTS, Contract, contract
from decode import decode
from schema import collection_window

```

```

def spine(tidy: pd.DataFrame, c: Contract) -> pd.DatetimeIndex:
    """Every sample instant from the first continuous sample to the last.

    Starting at the earliest event_time overall would start at a lagged
    analyser timestamp, before the first message, where no continuous
    instrument can have a value."""
    times = tidy.loc[tidy["tag"].isin(c.continuous), "event_time"]
    return pd.date_range(times.min(), times.max(), freq=pd.Timedelta(seconds=c.sample_seconds),
                          unit="ms", name="event_time")

```

```

def widen(tidy: pd.DataFrame, c: Contract, instants: pd.DatetimeIndex) -> pd.DataFrame:
    """Spine first, then the long table pivoted onto it: one row per instant,
    one column per birth tag, a hole wherever nothing measured survived."""
    wide = tidy.pivot(index="event_time", columns="tag", values="value")
    grid = pd.DataFrame(index=instants).join(wide, how="left")
    return grid.reindex(columns=c.names) # a tag that never reported still gets a column

```

```

def schedules(readings: pd.DataFrame, c: Contract, start: pd.Timestamp) -> pd.DataFrame:
    """Each analyser's analysis instants, read off the event times it reported:
    a period in samples and a phase relative to the grid start."""
    rows = []
    step = pd.Timedelta(seconds=c.sample_seconds)
    for t in c.tags:
        if t.get("analyserIntervalHours") is None:
            continue
        period = round(t["analyserIntervalHours"] * 3600 / c.sample_seconds)
        times = readings.loc[readings["tag"] == t["name"], "event_time"].drop_duplicates()
        phases = ((times - start) / step).round().astype(int) % period.value_counts()
        rows.append({"tag": t["name"], "period": period, "phase": int(phases.index[0]),
                     "off_phase_times": int(phases.iloc[1:].sum())})
    return pd.DataFrame(rows).set_index("tag")

```

```

def empty_cells(grid: pd.DataFrame, readings: pd.DataFrame, quarantine: pd.DataFrame,
                c: Contract) -> pd.DataFrame:
    """One row per empty cell before filling, with the reason it is empty."""
    sched = schedules(readings, c, grid.index[0])
    cells = grid.isna().rename_axis(columns="tag").stack()
    cells = cells[cells].reset_index()[["event_time", "tag"]]
    cells["row"] = grid.index.get_indexer(cells["event_time"])
    cells["tag_class"] = cells["tag"].map(lambda t: "analyser" if t in sched.index else "continuous")

    delivered = pd.MultiIndex.from_frame(readings[["tag", "event_time"]])

```

```

held = pd.MultiIndex.from_frame(quarantine[["tag", "event_time"]])
sampled = set(readings.loc[readings["tag"].isin(c.continuous), "event_time"])
key = pd.MultiIndex.from_frame(cells[["tag", "event_time"]])

period = cells["tag"].map(sched["period"])
phase = cells["tag"].map(sched["phase"])
on_schedule = cells["tag_class"].eq("continuous") | ((cells["row"] - phase) % period == 0)
first_value = grid.notna().idxmax().where(grid.notna().any())
leading = cells["event_time"] < cells["tag"].map(first_value)

cause = pd.Series("other", index=cells.index)
cause[~key.isin(delivered)] = "reading never delivered"
cause[~cells["event_time"].isin(sampled)] = "sample never delivered (lost message)"
cause[~on_schedule] = "between analyses"
cause[key.isin(held)] = "quarantined"
cells["cause"] = cause
cells["leading"] = leading.to_numpy()
return cells

def fill(grid: pd.DataFrame, tidy: pd.DataFrame, c: Contract) -> tuple[pd.DataFrame, pd.DataFrame, dict]:
    """Fill every hole. Returns (filled grid, mask of the cells that were empty, counts)."""
    mask = grid.isna()
    filled = grid.copy()

    # Continuous instruments: every hole is one missing sample between two
    # measured ones, so interpolate in time. Only an edge, which has a
    # neighbour on one side only, takes the nearest measured value.
    inside = filled[c.continuous].interpolate(method="time", limit_area="inside")
    filled[c.continuous] = inside.ffill().bfill()

    # Analysers: hold the last analysis until the next one, as the plant does.
    # Interpolating would use an analysis that had not happened yet. The first
    # row is seeded with the last analysis before the grid starts, if any, so
    # the leading holes are carried forward rather than filled from the future.
    analysers = filled[c.analysers]
    earlier = tidy[~tidy["tag"].isin(c.analysers) & (tidy["event_time"] < grid.index[0])]
    seed = earlier.sort_values("event_time").groupby("tag")["value"].last()
    first = analysers.iloc[0].isna()
    analysers.iloc[0] = analysers.iloc[0].fillna(seed)
    held = analysers.ffill()
    filled[c.analysers] = held.bfill()      # only a tag with no earlier value at all

    counts = {
        "continuous_interpolated": int((mask[c.continuous] & inside.notna()).sum().sum()),
        "continuous_edge_nearest": int((mask[c.continuous] & inside.isna()).sum().sum()),
        "analyser_seeded_from_before_grid": int((first & analysers.iloc[0].notna()).sum()),
        "analyser_held_forward": int((mask[c.analysers] & held.notna()).sum().sum()),
        "analyser_filled_backward": int((mask[c.analysers] & held.isna()).sum().sum()),
        "still_empty": int(filled.isna().sum().sum()),
    }
    return filled, mask, counts

def run(c: Contract, readings: pd.DataFrame, tidy: pd.DataFrame, quarantine: pd.DataFrame) -> dict:
    instants = spine(tidy, c)
    grid = widen(tidy, c, instants)
    off_grid = tidy[~tidy["event_time"].isin(instants)]
    cells = empty_cells(grid, readings, quarantine, c)
    filled, mask, fill_counts = fill(grid, tidy, c)

    summary = {"rows": len(grid), "start": str(instants[0]), "end": str(instants[-1]),
               "tidy_rows_off_grid": len(off_grid),
               "off_grid_before_start": int((off_grid["event_time"] < instants[0]).sum()),
               "fill": fill_counts}
    by_class = {}
    for label, tags in (("continuous", c.continuous), ("analyser", c.analysers)):

```

```

total = len(grid) * len(tags)
empty = int(grid[tags].isna().sum().sum())
by_class[label] = {"tags": len(tags), "cells": total, "empty": empty,
                  "pct": round(100 * empty / total, 2)}
summary["empty_before_fill"] = by_class

causes = (cells.groupby(["tag_class", "cause"]).agg(cells=("tag", "size"), leading=("leading", "sum"))
          .reset_index())

# All value columns first, then all masks: the grader pairs a mask with a
# tag by prefix, so XMEAS_1 must meet XMEAS_10_filled before XMEAS_10_filled.
table = pd.concat([filled, mask.add_suffix("_filled")], axis=1).reset_index()
PROCESSED.mkdir(exist_ok=True)
RESULTS.mkdir(exist_ok=True)
table.to_parquet(PROCESSED / "grid.parquet", index=False)
causes.to_csv(RESULTS / "empty_cells.csv", index=False)
(RESULTS / "grid_counts.json").write_text(json.dumps(summary, indent=2) + "\n", encoding="utf-8")
return {"grid": grid, "filled": filled, "mask": mask, "cells": cells, "causes": causes,
        "summary": summary, "schedules": schedules(readings, c, instants[0]),
        "readings": readings, "quarantine": quarantine, "tidy": tidy, "contract": c}

def main() -> dict:
    c = contract()
    readings = decode(show_plan=False).to_pandas()
    tidy, quarantine, _ = clean(readings, c.birth, collection_window(c, readings["received_at"]))
    return run(c, readings, tidy, quarantine)

if __name__ == "__main__":
    out = main()
    print(json.dumps(out["summary"], indent=2))
    print("\n" + out["causes"].to_string(index=False))

```

Code: src\idempotence.py

```

from __future__ import annotations

import hashlib
import os
import subprocess
import sys

from contract import PROCESSED, RESULTS, ROOT, raw_files

REPORT = RESULTS / "idempotence.txt"

def sha256(path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()

def outputs() -> dict[str, str]:
    files = sorted(p for p in [*PROCESSED.glob("*"), *RESULTS.glob("*")] if p.is_file() and p != REPORT)
    return {p.relative_to(ROOT).as_posix(): sha256(p) for p in files}

def run_pipeline() -> None:
    """The whole pipeline, in a fresh interpreter, as a person would run it."""
    done = subprocess.run([sys.executable, str(ROOT / "src" / "pipeline.py")], cwd=ROOT,
                          capture_output=True, text=True, encoding="utf-8",
                          env={**os.environ, "PYTHONIOENCODING": "utf-8"})
    if done.returncode:
        raise SystemExit(done.stdout + done.stderr)

def main() -> str:

```

```

raw_before = {p.name: sha256(p) for p in raw_files()}
run_pipeline()
first = outputs()
run_pipeline()
second = outputs()
raw_after = {p.name: sha256(p) for p in raw_files()}

lines = ["idempotence: src/pipeline.py run twice on the same raw capture", "",
        "  run 2 vs run 1  sha256 (first 16)  file"]
for name in sorted(set(first) | set(second)):
    a, b = first.get(name), second.get(name)
    lines.append(f"  {'same' if a == b else 'CHANGED'}           {(b or a or '-')[:16]}  {name}")
identical = first == second
lines += [
    f"raw {name}: sha256 {raw_before[name][:16]} before, "
    f"{'unchanged' if raw_after.get(name) == raw_before[name] else 'CHANGED'} after" for name in
    raw_before,
    f"all {len(second)} outputs byte-identical across the two runs: {'yes' if identical else
    'No'}"]
report = "\n".join(lines)
REPORT.write_text(report + "\n", encoding="utf-8")
if not identical or raw_before != raw_after:
    raise SystemExit(report + "\n\nNOT IDEMPOTENT")
return report

```

```

if __name__ == "__main__":
    print(main())

```

REPORT: REPORT.md

```

# Report
Yihao Yin (yihaoyin)

## Part 1 The Three Clocks
### Results
| messages | tags | timestamp minus sourceTimestamp (plant s) |
|---|---|---:|
| live (445) | 34 continuous | 0 |
| live | 14 analysers, every 0.1 h | 0 or 180 |
| live | 5 analysers, every 0.25 h | 0 to 720 |
| replayed, isHistorical (129) | all 53 | 1620 to 7200, median 3960 |
### Discussion
Windows are keyed on sourceTimestamp, the event time, because it is the only clock that says when the plant
was in the state a value describes; the message timestamp says when a batch was assembled and
receivedAt when this laptop saw it. 402 of the 443 message timestamps carry more than one event time.
The largest gap, 7200 plant s, is on a replayed message: the link dropped four times, for 32 to 46 wall
seconds (77 to 111 plant minutes), and each backlog was resent stamped with the time of resending. On
live messages the gap comes from the analysers, which republish an analysis unchanged until the next
one, so the lag reaches analyserIntervalHours x 3600 - 180, 720 s for the 0.25 h analysers.
receivedAt is on another machine: converted to plant time it lands a median 31 plant s before the live
timestamp, so this laptop runs about 0.2 wall s behind the publisher.

## Part 2 What You Collected
### Results
| item | count |
|---|---:|
| messages | 575 (574 telemetry, 1 birth), 28.95 plant hours |
| readings | 30422 |
| Good / Uncertain / Bad | 30337 / 70 / 15 |
| duplicates removed | 6333 |
| rejected by the schema | 3 |
| quarantined rows | 88 (70 Uncertain, 15 Bad, 3 value_in_unit_range) |
| tidy rows | 24001 |
| late readings | 5356 (22.32%) |
### Discussion

```

The pipeline quarantines, then deduplicates, then validates. A status belongs to a delivery, not a measurement: 28 analyser values arrived Good in one message and Uncertain or Bad in another, with the same value. So each non-Good delivery is quarantined on its own, and the Good rows are deduplicated on (tag, event_time), keeping the first arrival; seq wraps every 256 messages and is not an identity. The 6333 duplicates are 6266 analyser republishes and 67 rows of two messages delivered twice.

The pandera schema is generated from the birth tags (names, and a range per unit), plus a known status set and an event_time inside the collection window. With lazy=True it rejects 3 of the 24004 Good measurements, XMEAS_37 product D at -0.0032 to -0.0017 mole %: analyser noise near zero, but not physical. The Uncertain readings all look plausible, so the status decides, not the value.

The gate test breaks 5 of 300 copied rows (undeclared tag, out-of-range value, missing value, unknown status, wrong year); each is caught by its own check and no clean row is flagged. Two runs of src/pipeline.py give 14 byte-identical outputs, so the pipeline is idempotent.

Part 3 What You Filled

Results

```
| tags | cells | empty before filling | cause | fill |
|---|---:|---:|---|---|
| 34 continuous | 19584 | 192 (0.98%) | 136 in 4 samples never delivered, 56 quarantined | linear
  interpolation in time |
| 19 analysers | 10944 | 6340 (57.93%) | 6337 between analyses, 3 quarantined | last analysis held forward |
```

Discussion

The grid has 576 rows every 180 plant s, from 02:49:54 on 2029-07-20 to 07:34:54 the next day. It starts at the first continuous sample, not at the earliest event time, a lagged analyser timestamp that would open the grid with rows no continuous tag could fill. The spine is built first and the pivot joined onto it.

Each continuous hole, from the four samples never delivered (04:49:54, 11:04:54, 21:22:54, 21:46:54) or the 56 quarantined readings, is one sample between two measured ones, so it is interpolated in time.

Quarantine runs first, so no Bad or Uncertain value is ever used.

The analyser holes are by design: 14 analysers report every second row and 5 every fifth, and $(14 \times 1/2 + 5 \times 4/5) / 19 = 57.9\%$. An analysis is the latest composition known until the next one, so it is held forward, as the plant holds it; interpolation would use an analysis that had not happened yet. The 5 holes in the first row take the 02:37:54 analysis from just before the grid, so nothing comes from the future. Boolean '<tag>_filled' columns mark all 6532 filled cells.

Part 4 The Watermark

Results

```
| allowance (plant min) | wall s | complete % | dropped as late | still waiting |
|---:|---:|---:|---:|---:|
| 0 | 0 | 77.68 | 5356 | 0 |
| 15 | 6.25 | 77.68 | 5356 | 202 |
| 30 | 12.5 | 78.16 | 5241 | 419 |
| 60 | 25 | 87.61 | 2974 | 837 |
| 90 | 37.5 | 96.65 | 805 | 1256 |
| 120 | 50 | 100.00 | 0 | 1676 |
| 240 | 100 | 100.00 | 0 | 3353 |
```

Discussion

A reading is late by how far its event_time is behind the newest event_time of any message that arrived before it, in line order, since one replay burst can share a receivedAt millisecond. All 5356 late readings came from the four replays (median 66, max 108 plant minutes); no live reading was late.

I would set the allowance to 120 plant minutes, 50 wall seconds. It is the smallest that admits everything here, with 12 minutes to spare, and close to the p95 lateness (117) the assignment reports for 6000 messages, so in a longer run under 1% of readings would miss; re-emitting their windows is cheaper than holding every window for the 198-minute worst case. Too short loses whole outages, not stragglers: at 30 minutes 22% miss, and the windows over each outage close nearly empty yet final. Too long delays every answer: at 240 minutes a disturbance is reported four plant hours late, with 3353 readings still waiting at the end, twice as many as at 120.

Part 5 Event Time vs. Processing Time

Results

Readings per window and the mean reactor pressure (XMEAS_7, kPa), over 30-minute windows on event_time and on receivedAt converted to plant time. Replayed readings in brackets.

```
| window, 2029-07-20 | readings, event_time | readings, receivedAt | mean, event_time | mean, receivedAt |
|---|---:|---:|---:|---:|
| 08:30 | 419 | 0 | 2700.70 | - |
| 09:30 | 419 | 338 (251) | 2714.11 | 2699.96 |
| 10:00 | 419 | 1541 (1122) | 2711.06 | 2709.70 |
```

